

Trabajo Práctico — Filtro de Sobel para detección de bordes

Sistemas Paralelos — UNTDF

Fechas límite de entrega:

- 7 de mayo de 2026: versiones secuencial, numba paralelo (CPU) y numpy.
- 21 de mayo de 2026: versión numba GPU.
- 28 de mayo de 2026: versiones PyTorch CPU y PyTorch GPU.

Este trabajo práctico integrador tiene como objetivo explorar el rendimiento de distintas implementaciones de algoritmos de alta demanda computacional. Dentro del campo de la computación científica, la interpretación y procesamiento de imágenes es una tarea común que a menudo requiere grandes cantidades de cálculos, especialmente cuando se trata de aplicar filtros complejos.

Se abordará el filtro de Sobel, que se utiliza para detectar bordes en imágenes. La multiplicación de matrices es un componente clave en este proceso, y su eficiencia puede marcar una gran diferencia en el rendimiento general.

El operador Sobel calcula el gradiente de la intensidad de una imagen en cada punto (píxel). Así, para cada punto, este operador da la magnitud del mayor cambio posible en la intensidad de la imagen, y la dirección de ese cambio. Para esto, se utilizan dos máscaras de convolución llamadas G_x y G_y , una para detectar cambios horizontales y otra para detectar cambios verticales.

Matemáticamente, el gradiente de una función de dos variables (en este caso, la función de intensidad de la imagen) para cada punto es un vector bidimensional cuyos componentes están dados por las primeras derivadas de las direcciones verticales y horizontales. Para cada punto de la imagen, el vector gradiente apunta en dirección del incremento máximo posible de la intensidad, y la magnitud del vector gradiente corresponde a la cantidad de cambio de la intensidad en esa dirección.

En términos de cálculo, si la intensidad es $I(x, y)$, el gradiente es $\nabla I = (\partial I / \partial x, \partial I / \partial y)$. En imágenes digitales esas derivadas se aproximan con diferencias finitas: por ejemplo, $\partial I / \partial x \approx I(x+1, y) - I(x-1, y)$, lo que explica la estructura básica $[-1, 0, 1]$ (restar izquierda y derecha para detectar cambios horizontales). Sobel extiende esta idea con las máscaras G_x y G_y , que combinan derivación y un leve suavizado local (pesos $1-2-1$), y permite estimar la magnitud del borde como $|\nabla I| \approx \sqrt{G_x^2 + G_y^2}$.

En Python, esto se implementa tomando para cada píxel su vecindad 3×3 , aplicando G_x y G_y (producto elemento a elemento y suma), obteniendo g_x y g_y , y combinándolos para la magnitud final; luego ese resultado se normaliza o recorta al rango de intensidades para visualizar bordes correctamente.

Para este trabajo, Sobel se aplicará sobre imágenes en escala de grises (1 canal). Si la imagen original está en RGB (3 canales), primero se debe convertir a una única intensidad por píxel, por ejemplo con luminancia: $I = 0.299R + 0.587G + 0.114B$. Esta conversión (de 3 canales a 1) debe respetar el enfoque de cada implementación: en secuencial, sin librerías vectorizadas; en NumPy/Numba/PyTorch, con las operaciones propias de cada framework. El criterio matemático de conversión debe mantenerse equivalente para que la comparación de tiempos y del porcentaje de píxeles blancos sea consistente.

Código breve para convertir de RGB a grises (3 canales a 1) en versión secuencial pura:

```
# rgb: lista 2D de pixeles, cada pixel como tupla/lista (R, G, B)
gray = []
for row in rgb:
    gray_row = []
    for r, g, b in row:
        i = int(0.299 * r + 0.587 * g + 0.114 * b)
        gray_row.append(0 if i < 0 else 255 if i > 255 else i)
    gray.append(gray_row)
```

python

Código breve equivalente en NumPy (vectorizado):

```
import numpy as np

# rgb: arreglo HxWx3 con canales R, G, B
rgb = np.asarray(img_rgb, dtype=np.float32)

gray = 0.299 * rgb[:, :, 0] + 0.587 * rgb[:, :, 1] + 0.114 * rgb[:, :, 2]
gray_u8 = np.clip(gray, 0, 255).astype(np.uint8)
```

python

Ejemplo breve en Python (Sobel sobre una matriz 2D):

```
import numpy as np

img = np.array([
    [10, 10, 10, 10],
    [10, 80, 80, 10],
    [10, 80, 80, 10],
    [10, 10, 10, 10],
], dtype=np.float32)

Gx = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=np.float32)
Gy = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]], dtype=np.float32)

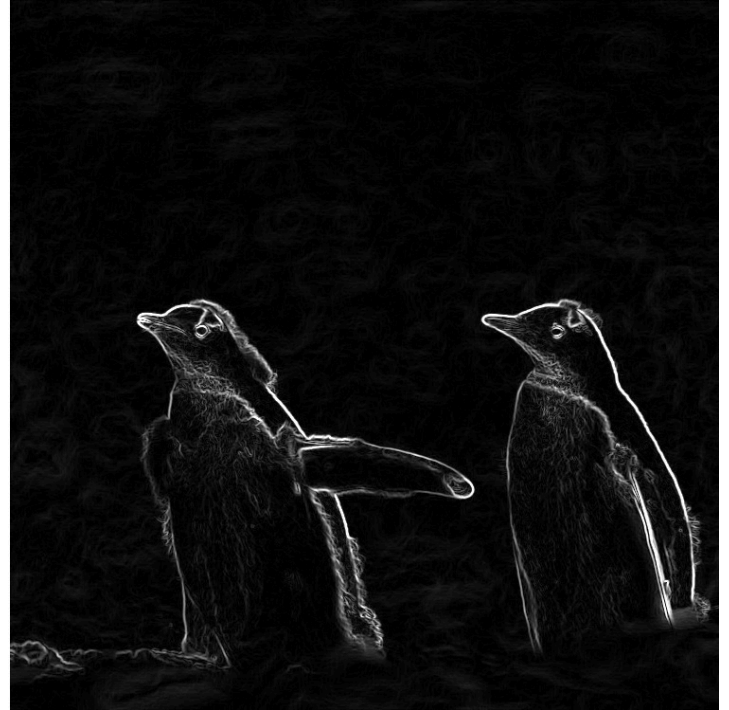
p = img[0:3, 0:3] # vecindad 3x3 de ejemplo
gx = np.sum(p * Gx)
```

python

```
gy = np.sum(p * Gy)
grad = np.sqrt(gx * gx + gy * gy)

print(f"gx={gx:.2f}, gy={gy:.2f}, |grad|={grad:.2f}")
```

Comparación visual entre la imagen original y el resultado del filtro Sobel:



Consigna

Se requiere implementar y comparar el rendimiento de distintas versiones del algoritmo de Sobel, utilizando diferentes enfoques de paralelización y optimización, incluyendo:

- secuencial
- numba paralelo (CPU)
- numpy
- numba GPU
- pytorch cpu
- pytorch gpu

1. Ejecución del benchmark

Ejecutar todos los scripts con el máximo posible de paralelismo que la librería y el hardware permitan, para cada una de las siguientes combinaciones de tamaño de imagen:

- 750x750
- 1500x1500

- 3000x3000
- 6000x6000

En cada ejecución, se deberán medir por separado: (1) tiempo de conversión RGB→gris y (2) tiempo de aplicación de Sobel. Además, se deberá reportar también el tiempo total como suma de ambos.

La medición de tiempos deberá considerar solo cómputo (conversión y Sobel), excluyendo carga y guardado en disco (I/O).

Para cada método y tamaño de imagen, se deberán realizar al menos 5 corridas y reportar el tiempo promedio de cada columna temporal (RGB→gris, Sobel y total).

Como métrica de salida se utilizará únicamente el porcentaje de píxeles blancos (valor 255) de la imagen Sobel: $\% \text{ blancos} = (\text{cantidad de píxeles con valor 255} / \text{cantidad total de píxeles}) * 100$.

2. Formato de presentación de resultados

El trabajo deberá ser presentado en formato de paper, de forma similar a los trabajos anteriores. La tabla de resultados deberá incluir, para cada combinación ejecutada, las siguientes columnas: método, tiempo RGB→gris (s), tiempo Sobel (s), tiempo total (s), % blancos, speed-up y performance (%). Armar una tabla por cada tamaño de imagen.

Para mantener la comparación consistente, se tomará el tiempo total (s) como referencia para calcular speed-up y performance (%). Los valores de tiempo, speed-up y performance (%) deberán calcularse utilizando los tiempos promedio de las corridas.

3. Análisis

Responder, siendo breve, tres preguntas por cada entrega:

Entrega 1 (secuencial, numba paralelo CPU y numpy)

1. ¿Qué diferencias de tiempo se observan entre secuencial, NumPy y Numba paralelo CPU para cada tamaño de imagen?
2. ¿Cómo evoluciona el speed-up de NumPy y Numba CPU respecto de secuencial al pasar de 750x750 a 6000x6000?
3. ¿Los resultados son equivalentes en términos de salida visual y porcentaje de píxeles blancos? En caso de diferencias, ¿a qué se atribuyen?

Entrega 2 (numba GPU)

1. ¿Qué mejora de tiempo se observa entre Numba GPU y Numba paralelo CPU para cada tamaño de imagen?
2. ¿En qué tamaños de imagen se amortiza el costo de transferencia de datos CPU↔GPU?

3. ¿El comportamiento del speed-up de Numba GPU es consistente al aumentar la resolución, o presenta umbrales/cambios de tendencia?

Entrega 3 (PyTorch CPU y PyTorch GPU)

1. ¿Qué diferencias de rendimiento se observan entre PyTorch CPU y PyTorch GPU para cada tamaño de imagen?
2. ¿Cómo se comparan PyTorch CPU/PyTorch GPU frente a las mejores implementaciones obtenidas en las entregas anteriores?
3. ¿Las salidas de PyTorch (CPU/GPU) son consistentes con Numba/NumPy en términos de bordes detectados y porcentaje de píxeles blancos, y qué factores podrían explicar diferencias?

4. Entorno y hardware

Al inicio del informe, incluir:

- CPU (modelo, núcleos físicos y lógicos)
- RAM disponible
- Sistema operativo y versión de Python
- Si se dispone de GPU (CUDA o MPS): se deberán reportar también los resultados de PyTorch y comentar las diferencias.

Formato de entrega

- Un único archivo **PDF o Markdown**.
- Nombre del archivo: `apellido_nombre_sobel.pdf` (o `.md`).
- Enviar por email.